第8章 图像压缩

- 算术编码

算术编码

假设某个字符的出现概率为 80%，该字符事实上只需要 $-\log_2(0.8) = 0.322$ 个二进制位进行编码

难道真的能只输出 0.322 个 0 或 0.322 个 1 吗？

算术编码的输出是：一个小数

算术编码对整条信息（无论信息有多么长），其输出仅仅是一个数，而且是一个介于0和1之间的二进制小数。
例如算术编码对某条信息的输出为1010001111，那么它表示小数0.1010001111，也即十进制数0.64

算术编码

算术编码具体方法是将被编码的信源消息表示成实数轴0-1之间的一个间隔，消息越长，编码表示的间隔就越小，即这一间隔所需的二进制位数就越多。

采用算术编码每个符号的平均编码长度可以为小数。

算术编码

设待编码的数据序列为“dacab”， 信源中各符号出现的概率依次为P(a)=0.4，P(b)=0.2，P(c)=0.2， P(d)=0.2。

数据序列中的各数据符号在区间［0, 1］内的间隔（赋值范围）设定为*a*=［0, 0.4），*b*=［0.4, 0.6），*c*=［0.6, 0.8）， d［0.8, 1.0）

表示前一间隔的起始位置

新间隔的起始位置和结束位置

$$\begin{cases} Start_N = Start_B + Left_C \times L \\ End_N = Start_B + Right_C \times L \end{cases}$$

前一间隔的长度

当前编码符号的初始区间的左端和右端

算术编码

待编码的数据序列为“dacab”，信源中各符号出现的概率依次为P(a)=0.4，P(b)=0.2，P(c)=0.2，　P(d)=0.2。

数据序列中的各数据符号在区间［0，1］内的间隔（赋值范围）设定为:

a=［0, 0.4）

b=［0.4, 0.6）

c=［0.6, 0.8）

d=［0.8, 1.0]

算术编码

✓ 待编码的数据序列为“dacab”

a=［0，0.4），b=［0.4，0.6），c=［0.6，0.8），d［0.8，1.0）

$$\begin{cases} Start_N = Start_B + Left_C \times L \\ End_N = Start_B + Right_C \times L \end{cases}$$

输入d：其初始间隔为［0.8, 1.0)

输入a：其初始间隔为［0, 0.4)

“a”的取值范围应在前一符号间隔［0.8，1.0）的［0，0.4）子区间内

$$Start_N = 0.8 + 0 \times (1.0 - 0.8) = 0.8$$

$$End_N = 0.8 + 0.4 \times (1.0 - 0.8) = 0.88$$

“a”的实际编码区间在［0.8，0.88）之间

算术编码

✓ 待编码的数据序列为“dacab”

a=[0, 0.4)， b=[0.4, 0.6)， c=[0.6, 0.8)， d[0.8, 1.0)

$$\begin{cases} Start_N = Start_B + Left_C \times L \\ End_N = Start_B + Right_C \times L \end{cases}$$

输入c：其初始间隔为[0.6, 0.8)

“c”的取值范围应在前一符号间隔[0.8, 0.88)的[0.6, 0.8)子区间内

$$Start_N = 0.8 + 0.6 \times (0.88 - 0.8) = 0.848$$

$$End_N = 0.8 + 0.8 \times (0.88 - 0.8) = 0.864$$

“c”的实际编码区间在[0.848, 0.864)之间

算术编码

a=［0, 0.4）, b=［0.4, 0.6）, c=［0.6, 0.8）, d［0.8, 1.0）

$$\begin{cases} Start_N = Start_B + Left_C \times L \\ End_N = Start_B + Right_C \times L \end{cases}$$

输入a：其初始间隔为［0, 0.4)

“a”取值范围应在前一符号间隔［0.848, 0.864）的［0, 0.4）子区间内

$$Start_N = 0.848 + 0 \times (0.864 - 0.848) = 0.848$$

$$End_N = 0.848 + 0.4 \times (0.864 - 0.848) = 0.8544$$

“a”的实际编码区间在［0.848, 0.8544）之间

算术编码

a=［0，0.4），b=［0.4，0.6），c=［0.6，0.8），d［0.8，1.0）

$$\begin{cases} Start_N = Start_B + Left_C \times L \\ End_N = Start_B + Right_C \times L \end{cases}$$

输入b：其初始间隔为［0.4, 0.6)

“b”取值范围应在前一符号间隔［0.848，0.8544)的［0.4，0.6)子区间内

$$Start_N = 0.848 + 0.4 \times (0.8544 - 0.848) = 0.85056$$

$$End_N = 0.848 + 0.6 \times (0.8544 - 0.848) = 0.85184$$

“b”的实际编码区间在［0.85056，0.85184)之间

算术编码

数据序列“dacab”已被描述为一个实数区间［0.85056，0.85184］，在此区间内的任一实数值都惟一对应该数据序列。这样，就可以用一个实数表示这一数据序列。把区间［0.85056，0.85184］用二进制形式表示为［0.110110011011，0.110110100001］。

［0.110110011011，0.110110100001］。

0.1101101位于这个区间内并且其编码最短，故把其作为数据序列“dacab”的编码输出。不考虑“0.”，把1101101作为本例中的数据序列的算术编码。由此可见，数据序列“dacab”用7比特的二进制代码就可以表示，平均码长为1.4比特／字符。

自适应模型算术编码

一、自适应模型问题引入？

1、静态模型如何实现？

对信息 bccb 中只有两个字符，概率分布为 Pb = 0.5，Pc = 0.5。压缩中不必更新概率分布，每次区间的划分都按此分布。

	输出区间的下限	输出区间的上限
压缩前	0.0	1.0
输入 b	0.0	0.5
输入 c	0.25	0.5
输入 c	0.375	0.5
输入 b	0.375	0.4375

自适应模型算术编码

2、静态模型缺点（即为什么用自适应模型？）

①静态模型无法适应信息的多样性，必须消耗一定的空间保存静态模型统计出的概率分布

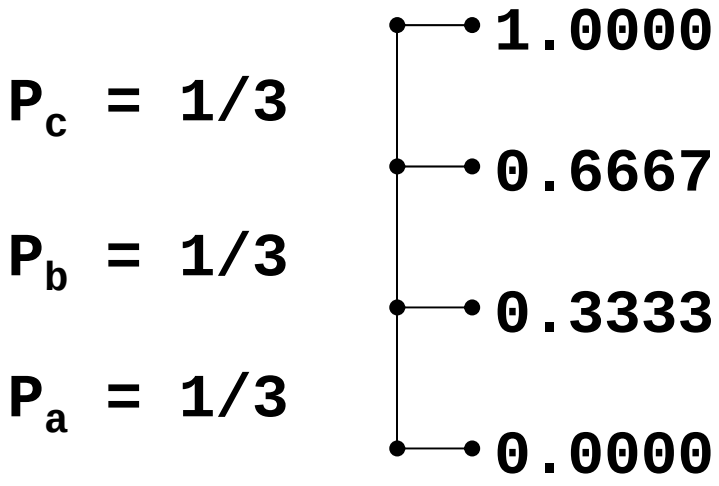
②静态模型需要在压缩前对信息内字符的分布进行统计，这一统计过程将消耗大量的时间，使得本来就比较慢的算术编码压缩更加缓慢。

③对较长的信息，静态模型统计出的符号概率是该符号在整个信息中的出现概率，而自适应模型可以统计出某个符号在某一局部的出现概率或某个符号相对于某一上下文的出现概率。

自适应模型算术编码

例：考虑某条信息中可能出现的字符仅有 a b c 三种，我们要压缩保存的原始信息为 bccb

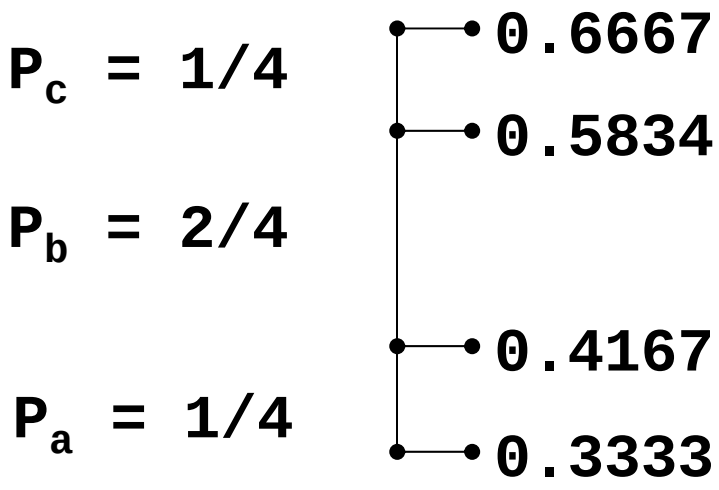
第一步：在没有开始压缩进程之前，假设我们对 a b c 三者在信息中的出现概率一无所知（我们采用的是自适应模型），即认为三者的出现概率相等，也就是都为1/3，我们将0-1区间按照概率的比例分配给三个字符，即a从0.0000到0.3333，b从0.3333到0.6667，c从0.6667到1.0000。用图形表示就是：



自适应模型算术编码

例：考虑某条信息中可能出现的字符仅有 a b c 三种，我们要压缩保存的原始信息为 bccb

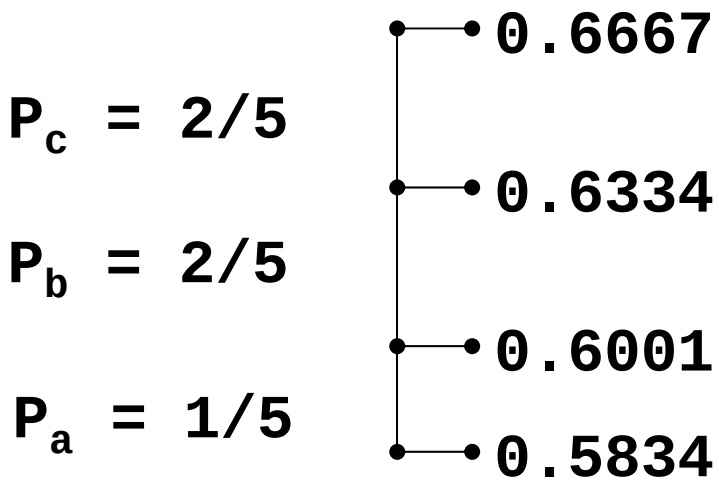
第二步：现在我们拿到第一个字符b，让我们把目光投向b对应的区间0.3333-0.6667。这时由于多了字符b，三个字符的概率分布变成：P_a=1/4，P_b=2/4，P_c=1/4。让我们按照新的概率分布比例划分0.3333-0.6667这一区间，划分的结果可以用图形表示为：



自适应模型算术编码

例：考虑某条信息中可能出现的字符仅有 a b c 三种，我们要压缩保存的原始信息为 bccb

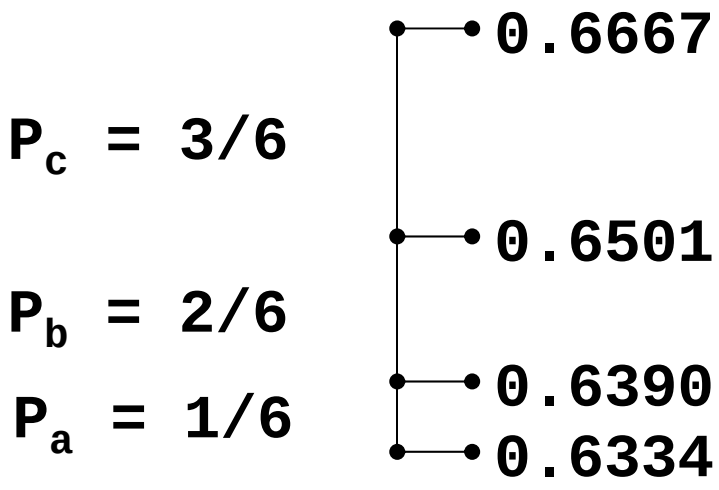
第三步：接着我们拿到字符c，我们现在要关注上一步中得到的c的区间0.5834-0.6667。新添了c以后，三个字符的概率分布变成P_a=1/5，P_b=2/5，P_c=2/5。我们用这个概率分布划分区间0.5834-0.6667：



自适应模型算术编码

例：考虑某条信息中可能出现的字符仅有 a b c 三种，我们要压缩保存的原始信息为 bccb

第四步：现在输入下一个字符c，三个字符的概率分布为：P_a=1/6，P_b=2/6，P_c=3/6。我们来划分c的区间0.6334-0.6667：



自适应模型算术编码

例：考虑某条信息中可能出现的字符仅有 a b c 三种，我们要压缩保存的原始信息为 bccb

第五步：输入最后一个字符b，因为是最后一个字符，不用再做进一步的划分了，上一步中得到的b的区间为0.6390-0.6501，让我们在这个区间内随便选择一个容易变成二进制的数，例如0.64，将它变成二进制0.1010001111，去掉前面没有太多意义的0和小数点，我们可以输出1010001111，这就是信息被压缩后的结果，我们完成了一次最简单的算术压缩过程



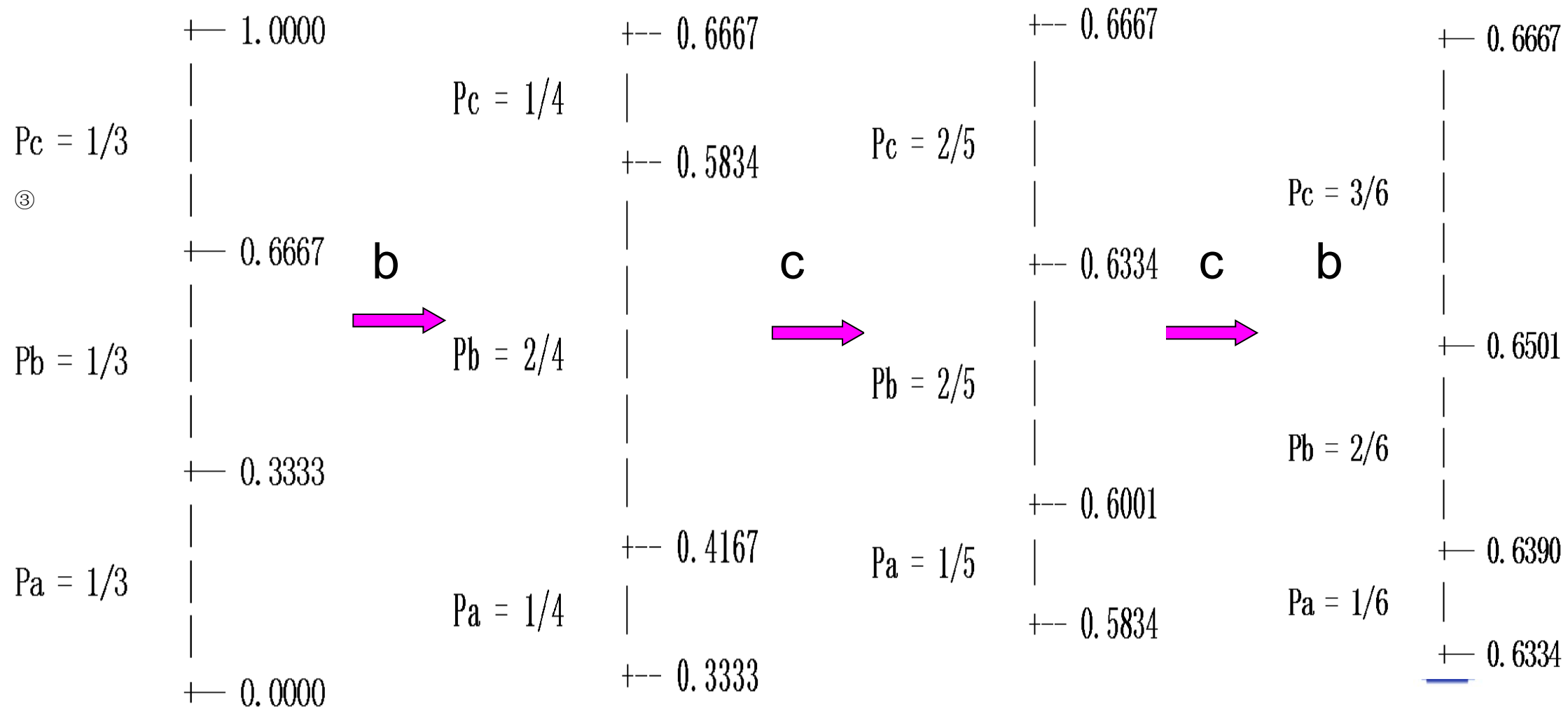
自适应模型算术编码

三、自适应算术编码如何解压缩呢？

解压缩之前我们仍设三个字符的概率相等，并得出第一幅分布图。先在二进制流数据前面加上 0 和小数点把它变成小数 0.1010001111，也就是十进制 0.64。发现 0.64 在分布图中落入字符 b 的区间内，我们立即输出字符 b，并得出三个字符新的概率分布。类似压缩时采用的方法，我们按照新的概率分布划分字符 b 的区间。在新的划分中0.64 落入了字符 c 的区间，则输出字符 c。同理类推，完成全部解压缩过程 。

自适应模型算术编码

如何解压缩呢？



自适应模型算术编码

- 模式选择直接影响效率，有固定模式和自适应模式。
- 自适应模式无需先定义概率模型，适合无法进行概率统计的信源，在这点上优越于哈夫曼编码。
- 在信源符号概率接近时，比哈夫曼编码效率高。
- 硬件实现比哈夫曼编码复杂。
- 在JPEG的扩展系统中被推荐代替哈夫曼编码。

字典压缩算法

- 字典编码方法是以类似查字典的方式进行编码，它的**基本原理是以较长的字符串或经常出现的字母组合构成字典中的各个词条**，并用相对较短的数字或符号来表示的方法。

- 字典编码按其构成的方式可分为**静态字典方法和动态字典方法**。

LZW 编码

LZW编码算法

- 1987年，Terry A. Welch在LZ78的基础上进行了改进，这就是著名的LZW压缩算法。

- LZW压缩算法是一种基于字典算法的编码方法。基本思想是建立一个编码表（转换表）也称串表，将输入字符串映射成定长的码字输出。

LZW编码算法

步骤1：将词典初始化为包含所有可能的单字字符，当前前缀P初始化为空。

步骤2：当前字符C:=字符流中的下一个字符。

步骤3：判断P＋C是否在词典中

(1) 如果“是”，则用C扩展P，即让P:=P＋C

(2) 如果“否”，则

输出与当前前缀P相对应的码字；
将P＋C添加到词典中；
令P:=C，现在的P仅包含一个字符C

步骤4：判断码字流中是否还有码字要译

(1) 如果“是”，就返回到步骤2；

(2) 如果“否” 把代表当前前缀P的码字输出到码字流； 结束。

LZW编码算法

举例：

- 原始码流 35 76 67 35 76 78 90 33 35 76 78 37

- 原理：
- 1.准备一个数据字典（可以看做一个数组），数组的前256项初始化为0,1,2，…，255，后面的项为空白。为方便起见，我们管字典的每一项叫“模式”。
- 2.开始对图像文件编码，图像文件从左向右扫描，把扫描得到的数据与字典中的模式进行比较。如果相同，就把这个“模式”index（在数组中的位置）座位码字输出。如果在字典中找不到与之匹配的模式，则在字典中创建一个新的模式（从第256项开始）。

LZW编码算法

原始码流 35 76 67 35 76 78 90 33 35 76 78 37

- **开始编码：**

a: 因为第一个数字是35，从字典中找到与之匹配的模式（也就是第35个），但我们不急着用第35个模式与之匹配，先看看第二个数字是76，希望在字典中能找到一个更长的模式“35 76”这样的模式，与之匹配，但不幸的是我们没有找到，所以只对第一个数字35编码，结果输出35；同时把“35 76”加入字典的第256项，希望以后能碰到它。

b: 同理，对76编码，输出76，同时把“76 67”加入字典的第257项。

c: 同理，对67编码，输出67，同时把“67 35”加入字典的第258项。

d: 这时注意了：对35编码，是不是现在还输出35呢？当然不是，我们发现35后面跟着76，扫描字典，可以发现第256个模式与之匹配，输出256.同时，将模式“35 76 78”加入到字典的第259项。

e: 接下来输出78，90，33，259，37。

所以输出的码流为 35 76 67 256 78 90 33 259 37。

LZW编码算法

□LZW编码举例

输入数据流：

位置	1	2	3	4	5	6	7	8	9
字符	A	B	B	A	B	A	B	A	C

编码过程：

步骤	码字	词典	输出
	1	A	
	2	B	
	3	C	
1	4	AB	1
2	5	B	2
3	6	BA	2
4	7	ABA	1
5	8	ABAC	7
6			

? ? ?

LZW编码算法

对LZW算法的分析

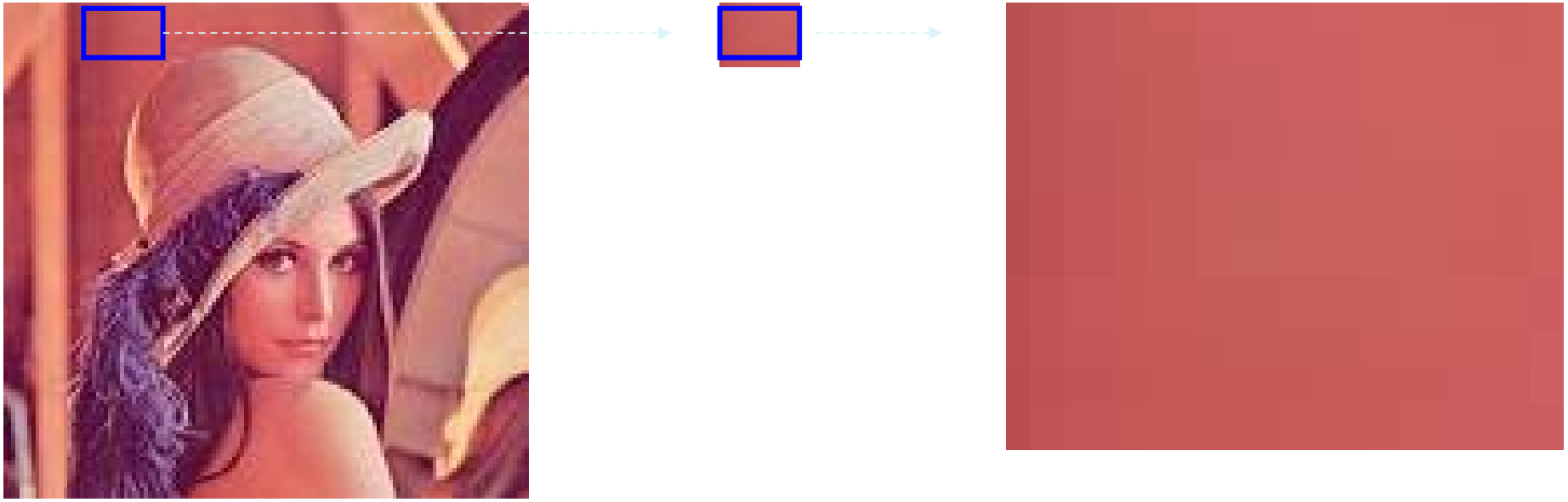
①由于LZW算法的关键是通过翻译表来实现压缩的，而且每次找出的串是在表中最长的串，然后对其进行编码，因此，如果我们设法增加表中长串的数量，则压缩比也就越高，即串越长，输出越少。

②串表中的每一个串均具有前缀特性，即：如果Kx(K为一个串，x为一个字符）在串表中，则字的前缀K一定在串表中（称之为前缀条件）。

行程 编码

游程长度编码（行程编码）

- 现实中有许多这样的图像，在一幅图像中具有许多颜色相同的图块。在这些图块中，许多行上都具有相同的颜色，或者在一行上有许多连续的像素都具有相同的颜色值。



游程长度编码（行程编码）

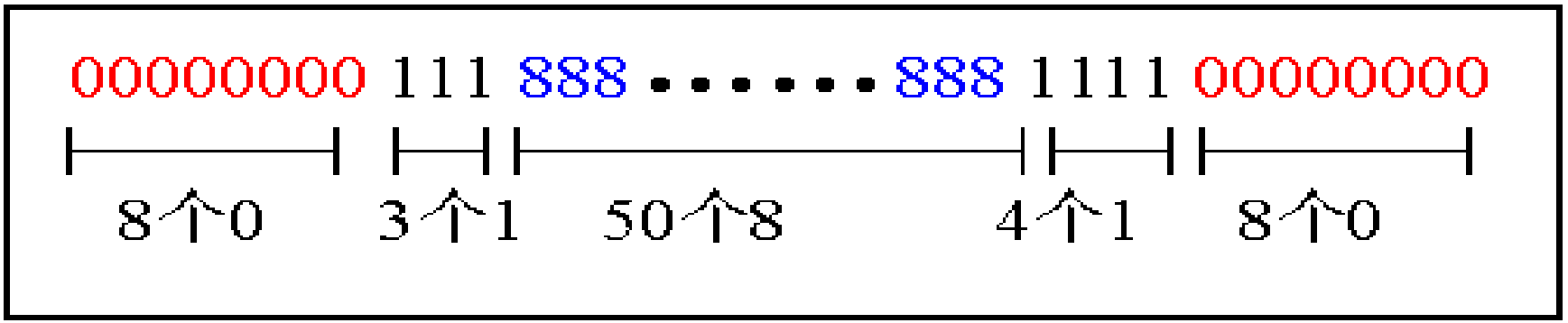
行程编码：不需要存储每一个像素的颜色值，而仅仅存储一**个像素的颜色值**以及**具有相同颜色的像素数目**就可以，或者存储一个像素的颜色值以及具有相同颜色值的行数。

行程长度：具有相同颜色并且是连续的像素数目。

游程长度编码（行程编码）

例子

- 假定一幅灰度图像，第n行的象素值为：



用RLC编码方法得到的代码为：

80315084180。

代码中用黑体表示的数字是行程长度，黑体字后面的数字代表象素的颜色值。

例如黑体字50代表有连续50个像素具有相同的颜色值，它的颜色值是8。

游程长度编码（行程编码）

特点

- 对比RLC编码前后的代码数可以发现，在编码前要用73个代码表示这一行的数据，而编码后只要用11个代码表示代表原来的73个代码，压缩前后的数据量之比约为7:1，即压缩比为7:1
- 这说明RLC确实是一种压缩技术，而且这种编码技术相当直观，也非常经济。
- 压缩比：RLC所能获得的压缩比有多大，这主要是取决于图像本身的特点。如果图像中具有相同颜色的图像块越大，图像块数目越少，获得的压缩比就越高。反之，压缩比就越小。
- 译码时按照与编码时采用的相同规则进行，还原后得到的数据与压缩前的数据完全相同。因此，RLC是无损压缩技术。

游程长度编码（行程编码）

分析：

- 对于有大面积色块的图象，压缩效果很好；
- 对于纷杂的图象，压缩效果不好，最坏情况下，会加倍图象。





总结：

本章主要目的是介绍压缩的必要性，以及数字图像压缩的理论基础知识，同时描述了一些常用压缩方法，包括霍夫曼编码、算数编码、LZW编码和行程编码。